



**UNIVERSIDAD AUTÓNOMA DE
CHIHUAHUA**

FACULTAD DE INGENIERÍA

Base de datos avanzadas

Grupo: 7CC2

4.24 Proyecto Final

Integrantes del equipo:

Dominik Eliel Armendariz Mimbela – 368065

Braulio Sebastian Porras Olivas - 344175

Angel Damian Villalobos Ochoa – 367987

Ingenieria en Ciencias de la Computacion

Chihuahua, Chih. *Fecha de entrega* – 26 de Mayo Del 2026

El siguiente programa es una aplicación de escritorio desarrollada en Python que permite administrar información de empleados y departamentos mediante una base de datos Neo4j.

La interfaz gráfica fue construida con CustomTkinter, lo que le da una apariencia más moderna, mientras que ttk.Treeview se utiliza para mostrar los registros en forma de tabla.

Este sistema implementa operaciones básicas de CRUD:

- Create: registrar empleados y departamentos.
- Read: consultar y buscar registros.
- Update: actualizar datos de empleados.
- Delete: eliminar empleados y, en algunos casos, también departamentos.

Además, el programa incluye validaciones para evitar errores en la captura de datos, como letras en campos numéricos, fechas mal formadas o registros duplicados.

Descripción general del funcionamiento

La aplicación se conecta a una base de datos Neo4j llamada blog. En esta base de datos se manejan dos tipos de nodos:

- Dept para representar departamentos.
- Emp para representar empleados.

Entre ambos nodos existe la relación:

- TRABAJA_EN

Esta relación indica en qué departamento trabaja cada empleado.

Al iniciar el programa, primero intenta conectarse a Neo4j.

Si la conexión es correcta, carga datos iniciales si la base está vacía y después muestra la información en pantalla.

Librerías utilizadas

re: Se utiliza para trabajar con expresiones regulares.

En este código sirve principalmente para validar el formato de las fechas.

customtkinter as ctk: Es la librería que permite crear una interfaz gráfica moderna basada en Tkinter.

Se usa para crear ventanas, botones, pestañas, etiquetas y campos de texto.

tkinter.ttk: Se utiliza para el componente Treeview, que permite mostrar registros en forma de tabla.

tkinter.messagebox: Se usa para mostrar ventanas emergentes con mensajes de error, advertencia, confirmación o éxito.

neo4j.GraphDatabase: Permite conectarse y ejecutar consultas sobre la base de datos Neo4j.

Configuración de conexión

El programa define los datos necesarios para conectarse a Neo4j:

- URI: dirección del servidor.
- USER: nombre de usuario.
- PASSWORD: contraseña.
- DB_NAME: nombre de la base de datos.

Luego se intenta crear el controlador de conexión driver.

Si la conexión falla, se muestra un error y el controlador queda en None.

```

import re
import customtkinter as ctk
from tkinter import ttk, messagebox
from neo4j import GraphDatabase

URI      = "neo4j://127.0.0.1:7687"
USER     = "neo4j"
PASSWORD = "dangos123"
DB_NAME  = "blog"

ctk.set_appearance_mode("Dark")
ctk.set_default_color_theme("blue")

try:
    driver = GraphDatabase.driver(URI, auth=(USER, PASSWORD))
    driver.verify_connectivity()
    print("Conexión exitosa")
except Exception as e:
    print(f"Error: {e}")
    driver = None

```

Datos iniciales

El diccionario DATOS_INICIALES contiene dos listas:

Departamentos

Incluye varios departamentos con los campos:

- deptno: número del departamento.
- dname: nombre del departamento.
- loc: ubicación.

Empleados

Incluye registros de empleados con campos como:

- empno: número de empleado.
- ename: nombre.
- job: puesto.

- mgr: nombre del jefe.
- hiredate: fecha de contratación.
- sal: salario.
- comm: comisión.
- dept: nombre del departamento al que pertenece.

Estos datos sirven para poblar la base de datos al inicio, siempre que esté vacía.

```
DATOS_INICIALES = {
  "departamentos": [
    {"deptno": 10, "dname": "CONTABILIDAD", "loc": "CIUDAD DE MEXICO"},
    {"deptno": 20, "dname": "INVESTIGACION", "loc": "GUADALAJARA"},
    {"deptno": 30, "dname": "VENTAS", "loc": "MONTERREY"},
    {"deptno": 40, "dname": "OPERACIONES", "loc": "CHIHUAHUA"},
  ],
  "empleados": [
    {"empno": 7839, "ename": "REYES", "job": "DIRECTOR", "mgr": None,
     "hiredate": "1981-11-17", "sal": 5000.0, "comm": 0.0, "dept": "CONTABILIDAD"},
    {"empno": 7698, "ename": "MENDOZA", "job": "GERENTE", "mgr": "REYES",
     "hiredate": "1981-05-01", "sal": 2850.0, "comm": 0.0, "dept": "VENTAS"},
    {"empno": 7782, "ename": "HERRERA", "job": "GERENTE", "mgr": "REYES",
     "hiredate": "1981-06-09", "sal": 2450.0, "comm": 0.0, "dept": "CONTABILIDAD"},
    {"empno": 7566, "ename": "RAMIREZ", "job": "GERENTE", "mgr": "REYES",
     "hiredate": "1981-04-02", "sal": 2975.0, "comm": 0.0, "dept": "INVESTIGACION"},
    {"empno": 7788, "ename": "TORRES", "job": "ANALISTA", "mgr": "RAMIREZ",
     "hiredate": "1982-12-09", "sal": 3000.0, "comm": 0.0, "dept": "INVESTIGACION"},
    {"empno": 7902, "ename": "VARGAS", "job": "ANALISTA", "mgr": "RAMIREZ",
     "hiredate": "1981-12-03", "sal": 3000.0, "comm": 0.0, "dept": "INVESTIGACION"},
    {"empno": 7499, "ename": "GUTIERREZ", "job": "VENDEDOR", "mgr": "MENDOZA",
     "hiredate": "1981-02-20", "sal": 1600.0, "comm": 300.0, "dept": "VENTAS"},
    {"empno": 7521, "ename": "CASTILLO", "job": "VENDEDOR", "mgr": "MENDOZA",
     "hiredate": "1981-02-22", "sal": 1250.0, "comm": 500.0, "dept": "VENTAS"},
    {"empno": 7654, "ename": "MORALES", "job": "VENDEDOR", "mgr": "MENDOZA",
     "hiredate": "1981-09-28", "sal": 1250.0, "comm": 1400.0, "dept": "VENTAS"},
    {"empno": 7844, "ename": "JIMENEZ", "job": "VENDEDOR", "mgr": "MENDOZA",
     "hiredate": "1981-09-08", "sal": 1500.0, "comm": 0.0, "dept": "VENTAS"},
    {"empno": 7369, "ename": "LOPEZ", "job": "EMPLEADO", "mgr": "VARGAS",
     "hiredate": "1980-12-17", "sal": 800.0, "comm": 0.0, "dept": "INVESTIGACION"},
    {"empno": 7876, "ename": "GARCIA", "job": "EMPLEADO", "mgr": "TORRES",
     "hiredate": "1983-01-12", "sal": 1100.0, "comm": 0.0, "dept": "INVESTIGACION"},
    {"empno": 7900, "ename": "FLORES", "job": "EMPLEADO", "mgr": "MENDOZA",
     "hiredate": "1981-12-03", "sal": 950.0, "comm": 0.0, "dept": "VENTAS"},
    {"empno": 7934, "ename": "CRUZ", "job": "EMPLEADO", "mgr": "HERRERA",
     "hiredate": "1982-01-23", "sal": 1300.0, "comm": 0.0, "dept": "CONTABILIDAD"},
  ]
}
```

Función inyectar_datos_iniciales()

Esta función revisa si ya existen empleados en la base de datos.

Si ya hay datos: No inserta nada y muestra un mensaje indicando cuántos empleados hay.

Si la base está vacía: Inserta primero los departamentos con MERGE, para evitar duplicados.

Después inserta los empleados con CREATE y crea la relación TRABAJA_EN entre cada empleado y su departamento.

Esta función permite que la aplicación empiece con información lista para consultar y probar.

```
def inyectar_datos_iniciales():
    try:
        with driver.session(database=DB_NAME) as session:
            total = session.run("MATCH (e:Emp) RETURN count(e) AS n").single()[0]
            if total > 0:
                print(f"BD ya tiene {total} empleado(s), no se inyectan datos.")
                return

            print("Inyectando datos iniciales...")
            for d in DATOS_INICIALES["departamentos"]:
                session.run("""
                    MERGE (dept:Dept {deptno: $deptno})
                    SET dept.dname = $dname, dept.loc = $loc
                    """, **d)

            for emp in DATOS_INICIALES["empleados"]:
                session.run("""
                    MATCH (d:Dept {dname: $dept})
                    CREATE (e:Emp {
                        empno: $empno, ename: $ename, job: $job,
                        mgr: $mgr, hiredate: $hiredate,
                        sal: $sal, comm: $comm
                    })
                    CREATE (e)-[:TRABAJA_EN]->(d)
                    """, **emp)

            print(f"{len(DATOS_INICIALES['empleados'])} empleados y {len(DATOS_INICIALES['departamentos'])} departamentos creados.")
    except Exception as e:
        print(f"Error al inyectar datos: {e}")
```

Funciones de validación

Estas funciones ayudan a que los datos ingresados sean correctos antes de guardarlos.

solo_letras(valor, campo): Verifica que un texto contenga solo letras y espacios. Se usa para nombres, puestos, jefes y nombres de departamentos.

validar_fecha(fecha_str): Comprueba que la fecha tenga el formato AAAA-MM-DD. También valida que año, mes y día estén dentro de un rango razonable.

validar_numerico(valor_str, campo): Verifica que un dato sea numérico y no negativo.

`validar_entero_positivo(valor_str, campo)`: Comprueba que un valor sea un número entero mayor que cero. Se usa para claves como EMPNO y DEPTNO.

`driver_ok()`: Verifica que exista conexión con Neo4j antes de ejecutar cualquier operación. Si no hay conexión, muestra un mensaje de error y evita que el programa siga.

```
def solo_letras(valor, campo):
    if valor and not valor.replace(" ", "").isalpha():
        messagebox.showerror("Error de validación", f"'{campo}' solo debe contener letras.\nEjemplo: GARCIA, VENTAS")
        return False
    return True

def validar_fecha(fecha_str):
    if not fecha_str: return True
    if not re.match(r"^\d{4}-\d{2}-\d{2}$", fecha_str):
        messagebox.showerror("Error de validación", "Fecha inválida. Usa el formato AAAA-MM-DD\nEjemplo: 1981-05-01")
        return False
    anio, mes, dia = map(int, fecha_str.split("-"))
    if not (1900 <= anio <= 2100) or not (1 <= mes <= 12) or not (1 <= dia <= 31):
        messagebox.showerror("Error de validación", "Fecha fuera de rango.\nVerifica año (1900-2100), mes (1-12) y día (1-31).")
        return False
    return True

def validar_numerico(valor_str, campo):
    if not valor_str or not valor_str.replace("$", "").strip():
        return True, 0.0
    try:
        valor = float(valor_str.replace("$", "").strip())
    except ValueError:
        messagebox.showerror("Error de validación", f"'{campo}' debe ser un número.\nEjemplo: 3000 o 500.50")
        return False, None
    if valor < 0:
        messagebox.showerror("Error de validación", f"'{campo}' no puede ser negativo.")
        return False, None
    return True, valor

def validar_entero_positivo(valor_str, campo):
    if not valor_str: return True, None
    try:
        valor = int(valor_str)
        if valor <= 0: raise ValueError
    except ValueError:
        messagebox.showerror("Error de validación", f"'{campo}' debe ser un número entero positivo.")
        return False, None
    return True, valor

def driver_ok():
    if not driver:
        messagebox.showerror("Sin conexión", "No hay conexión con Neo4j.\nReinicia la aplicación.")
        return False
    return True
```

Operaciones CRUD

1. alta_empleado()

Esta función registra un nuevo empleado.

Pasos que realiza:

1. Lee los datos capturados en la interfaz.

2. Convierte nombre, puesto y jefe a mayúsculas.
3. Valida que los campos obligatorios no estén vacíos.
4. Verifica que:
 - el nombre solo tenga letras,
 - el puesto solo tenga letras,
 - el jefe solo tenga letras,
 - salario y comisión sean numéricos,
 - EMPNO y DEPTNO sean enteros positivos,
 - la fecha tenga formato correcto.
5. Busca si el departamento existe usando el DEPTNO.
6. Verifica que no exista ya un empleado con el mismo nombre o número.
7. Si el jefe no existe, pregunta al usuario si desea continuar.
8. Crea el nodo Emp y la relación TRABAJA_EN.
9. Muestra un mensaje de éxito.
10. Limpia los campos y actualiza la tabla.

Esta función evita registros inválidos o repetidos.

2. alta_departamento()

Esta función registra un nuevo departamento.

Pasos que realiza:

1. Lee DEPTNO, DNAME y LOC.
2. Valida que los campos no estén vacíos.
3. Comprueba que el nombre y la localización tengan solo letras.
4. Verifica que DEPTNO sea un entero positivo.
5. Revisa que no exista otro departamento con el mismo nombre o número.
6. Si todo es correcto, crea el nodo Dept.
7. Limpia los campos y actualiza la tabla de departamentos.


```

def alta_empleado():
    if not driver_ok(): return

    ename = ent_name.get().strip().upper()
    job = ent_job.get().strip().upper()
    deptno_str = ent_dept.get().strip()
    mgr_str = ent_mgr.get().strip().upper()
    hiredate_str = ent_hiredate.get().strip()
    empno_str = ent_empno.get().strip()

    if not ename or not job or not deptno_str:
        messagebox.showwarning("Faltan datos", "Nombre, Puesto y DEPTNO del departamento son obligatorios.")
        return

    if not solo_letras(ename, "Nombre"): return
    if not solo_letras(job, "Puesto"): return
    if not solo_letras(mgr_str, "Jefe (MGR)": return

    ok, sal = validar_numerico(ent_sal.get(), "Salario")
    if not ok: return
    ok, comm = validar_numerico(ent_comm.get(), "Comisión")
    if not ok: return
    ok, empno = validar_entero_positivo(empno_str, "EMPNO")
    if not ok: return
    ok, deptno = validar_entero_positivo(deptno_str, "DEPTNO")
    if not ok: return
    if not validar_fecha(hiredate_str): return

    try:
        with driver.session(database=DB_NAME) as session:
            dept_node = session.run("MATCH (d:Dept {deptno:$deptno}) RETURN d.dname AS dname", deptno=deptno).single()
            if not dept_node:
                messagebox.showerror("Error FK", f"El departamento con DEPTNO '{deptno}' no existe.\nRegístralo primero en la pestaña 'Departamentos'")
                return

            dname = dept_node["dname"]

```

```

            if session.run("MATCH (e:Emp {ename:$ename}) RETURN e", ename=ename).single():
                messagebox.showerror("Duplicado", f"Ya existe un empleado llamado '{ename}'.")
                return
            if empno and session.run("MATCH (e:Emp {empno:$empno}) RETURN e", empno=empno).single():
                messagebox.showerror("Duplicado", f"Ya existe un empleado con EMPNO {empno}.")
                return

            if mgr_str and not session.run("MATCH (e:Emp {ename:$mgr}) RETURN e", mgr=mgr_str).single():
                if not messagebox.askyesno("Jefe no encontrado", f"No existe ningún empleado llamado '{mgr_str}'.\n¿Deseas registrar al empleado de nuevo?"):
                    return

            session.run("""
                MATCH (d:Dept {deptno: $deptno})
                CREATE (e:Emp {empno:$empno, ename:$ename, job:$job,
                    mgr:$mgr, hiredate:$hiredate, sal:$sal, comm:$comm})
                CREATE (e)-[:TRABAJA_EN]->(d)
            """, empno=empno, ename=ename, job=job, mgr=mgr_str or None,
                hiredate=hiredate_str or None, sal=sal, comm=comm, deptno=deptno)

            messagebox.showinfo("Éxito", f"Empleado '{ename}' registrado en el departamento '{dname}'.")
            limpiar_campos(); consulta_empleados()
    except Exception as e:
        messagebox.showerror("Error Neo4j", str(e))

def alta_departamento():
    if not driver_ok(): return

    deptno_str = ent_new_deptno.get().strip()
    dname = ent_new_dname.get().strip().upper()
    loc = ent_new_loc.get().strip().upper()

    if not deptno_str or not dname or not loc:
        messagebox.showwarning("Faltan datos", "DEPTNO, Nombre y Localización son obligatorios.")
        return

```

```

if not solo_letras(dname, "Nombre Depto"): return
if not solo_letras(loc, "Localización"): return

ok, deptno = validar_entero_positivo(deptno_str, "DEPTNO")
if not ok: return

try:
    with driver.session(database=DB_NAME) as session:
        if session.run("MATCH (d:Dept {dname:$dname}) RETURN d", dname=dname).single():
            messagebox.showerror("Duplicado", f"El departamento '{dname}' ya existe.")
            return
        if session.run("MATCH (d:Dept {deptno:$deptno}) RETURN d", deptno=deptno).single():
            messagebox.showerror("Duplicado", f"El DEPTNO '{deptno}' ya está en uso.")
            return

        session.run("""
            CREATE (d:Dept {deptno: $deptno, dname: $dname, loc: $loc})
            """, deptno=deptno, dname=dname, loc=loc)

        messagebox.showinfo("Éxito", f"Departamento '{dname}' registrado correctamente.")
        ent_new_deptno.delete(0, "end")
        ent_new_dname.delete(0, "end")
        ent_new_loc.delete(0, "end")
        consulta_departamentos()
except Exception as e:
    messagebox.showerror("Error", str(e))

```

3. consulta_empleados()

Esta función muestra todos los empleados en la tabla principal.

Hace lo siguiente:

- Limpia la tabla antes de cargar datos nuevos.
- Ejecuta una consulta Cypher para obtener empleados y su departamento.
- Muestra los resultados ordenados por nombre.
- Formatea salario y comisión con símbolo de dólar.
- Actualiza una etiqueta que muestra el total de empleados.

4. consulta_departamentos()

Esta función actualiza la tabla de departamentos.

Hace lo siguiente:

- Limpia la tabla de departamentos.
- Consulta todos los nodos Dept.
- Muestra DEPTNO, DNAME y LOC.

5. buscar_empleado()

Permite buscar empleados usando distintos criterios.

Criterios disponibles:

- Nombre
- EMPNO
- DEPTNO
- Salario
- Puesto

Funcionamiento:

1. Lee el valor escrito en la caja de búsqueda.
2. Toma el criterio seleccionado en el combo.
3. Construye dinámicamente la cláusula WHERE.
4. Ejecuta la consulta en Neo4j.
5. Muestra solo los registros encontrados.

Si no encuentra coincidencias, muestra un mensaje y vuelve a cargar todos los empleados.

6. cambio_empleado()

Esta función actualiza los datos de un empleado existente.

Funcionamiento:

1. Toma el nombre del empleado como dato principal.
2. Valida los campos que se desean modificar.
3. Verifica que el empleado exista.
4. Si se escribió un nuevo jefe, comprueba si también existe.
5. Usa COALESCE para actualizar solo los campos que tengan valor.
6. Muestra un mensaje si hubo cambios.
7. Refresca la tabla principal.

Esta función permite modificar solo algunos datos sin borrar el resto.

7. baja_empleado()

Elimina un empleado de la base de datos.

Funcionamiento:

1. Verifica que se haya escrito un nombre.
2. Busca el empleado y su departamento.
3. Comprueba si ese empleado es jefe de otras personas.
4. Si tiene subordinados, no permite eliminarlo.
5. Calcula si es el único empleado de su departamento.
6. Pide confirmación antes de borrar.
7. Si el departamento queda vacío, pregunta si también debe eliminarse.
8. Elimina el nodo del empleado con DETACH DELETE.
9. Si corresponde, elimina también el departamento.

Esta función evita inconsistencias, como borrar un jefe que todavía tiene empleados a su cargo.

```
def consulta_empleados():
    if not driver.ok(): return
    for row in tabla.get_children():
        tabla.delete(row)
    try:
        with driver.session(database=DB_NAME) as session:
            results = session.run("""
                MATCH (e:Emp)-[:TRABAJA_EN]->(d:Dept)
                RETURN e.empno AS empno, e.ename AS ename, e.job AS job,
                       e.mgr AS mgr, e.hiredate AS hiredate,
                       e.sal AS sal, e.comm AS comm,
                       d.deptno AS deptno, d.loc AS loc
                ORDER BY e.ename
            """)
            total = 0
            for r in results:
                tabla.insert("", "end", values=(
                    r["empno"] or "-", r["ename"], r["job"],
                    r["mgr"] or "-", r["hiredate"] or "-",
                    f"${r['sal']:.2f}", f"${(r['comm'] or 0.0):.2f}",
                    r["deptno"] or "-", r["loc"] or "-"
                ))
                total += 1
            lbl_total.configure(text=f"Total: {total} empleado(s)")
    except Exception as e:
        messagebox.showerror("Error", str(e))

def consulta_departamentos():
    if not driver.ok(): return
    for row in tabla_dept.get_children():
        tabla_dept.delete(row)
    try:
        with driver.session(database=DB_NAME) as session:
            results = session.run("MATCH (d:Dept) RETURN d.deptno AS deptno, d.dname AS dname, d.loc AS loc ORDER BY d.deptno")
            for r in results:
                tabla_dept.insert("", "end", values=(r["deptno"], r["dname"], r["loc"]))
```

```

except Exception as e:
    messagebox.showerror("Error", str(e))

def buscar_empleado():
    if not driver.ok(): return
    busqueda = ent_buscar.get().strip().upper()
    criterio = combo_criterio.get()

    if not busqueda:
        messagebox.showwarning("Faltan datos", "Escribe un valor para buscar.")
        return

    for row in tabla.get_children():
        tabla.delete(row)

    try:
        with driver.session(database=DB_NAME) as session:
            if criterio == "EMPNO":
                where_clause = "WHERE e.empno = toInteger($busqueda)"
            elif criterio == "Salario":
                where_clause = "WHERE e.sal = toFloat($busqueda)"
            elif criterio == "DEPTNO":
                where_clause = "WHERE d.deptno = toInteger($busqueda)"
            elif criterio == "Puesto":
                where_clause = "WHERE e.job CONTAINS $busqueda"
            else:
                where_clause = "WHERE e.ename CONTAINS $busqueda"

            query = f"""
                MATCH (e:Emp)-[:TRABAJA_EN]->(d:Dept)
                {where_clause}
                RETURN e.empno AS empno, e.ename AS ename, e.job AS job,
                       e.mgr AS mgr, e.hiredate AS hiredate,
                       e.sal AS sal, e.comm AS comm,
                       d.deptno AS deptno, d.loc AS loc
                ORDER BY e.ename
            """

```

```

        results = session.run(query, busqueda=busqueda)
        encontrado = False
        total = 0
        for r in results:
            encontrado = True
            total += 1
            tabla.insert("", "end", values=(
                r["empno"] or "-", r["ename"], r["job"],
                r["mgr"] or "-", r["hiredate"] or "-",
                f"${r['sal']:.2f}", f"${(r['comm'] or 0.0):.2f}",
                r["deptno"] or "-", r["loc"] or "-"
            ))
        if encontrado:
            lbl_total.configure(text=f"Resultados: {total}")
        else:
            messagebox.showinfo("Sin resultados", f"No se encontró ningún registro para {criterio}: '{busqueda}'.")
            consulta_empleados()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def cambio_empleado():
    if not driver_ok(): return

    ename = ent_name.get().strip().upper()
    if not ename:
        messagebox.showwarning("Faltan datos", "El campo Nombre es obligatorio para actualizar.")
        return

    job_val = ent_job.get().strip().upper() or None
    mgr_val = ent_mgr.get().strip().upper() or None
    hiredate_val = ent_hiredate.get().strip() or None

    if job_val and not solo_letras(job_val, "Puesto"): return
    if mgr_val and not solo_letras(mgr_val, "Jefe (MGR)": return
    ok, sal_val = validar_numerico(ent_sal.get(), "Salario")
    if not ok: return
    ok, comm_val = validar_numerico(ent_comm.get(), "Comisión")

```

```

    if not ok: return

    sal_val = sal_val if ent_sal.get().strip() else None
    comm_val = comm_val if ent_comm.get().strip() else None
    if not validar_fecha(hiredate_val): return

    try:
        with driver.session(database=DB_NAME) as session:
            if not session.run("MATCH (e:Emp {ename:$ename}) RETURN e", ename=ename).single():
                messagebox.showwarning("No encontrado", f"No existe ningún empleado llamado '{ename}'.")
                return

            if mgr_val and not session.run("MATCH (e:Emp {ename:$mgr}) RETURN e", mgr=mgr_val).single():
                if not messagebox.askyesno("Jefe no encontrado", f"No existe ningún empleado llamado '{mgr_val}'.\n¿Deseas guardar el cambio de toda
                    return

            result = session.run("""
                MATCH (e:Emp {ename:$ename})
                SET e.job = COALESCE($job, e.job),
                    e.mgr = COALESCE($mgr, e.mgr),
                    e.hiredate = COALESCE($hiredate, e.hiredate),
                    e.sal = COALESCE($sal, e.sal),
                    e.comm = COALESCE($comm, e.comm)
                RETURN e
            """, ename=ename, job=job_val, mgr=mgr_val, hiredate=hiredate_val, sal=sal_val, comm=comm_val)

            if result.consume().counters.properties_set > 0:
                messagebox.showinfo("Éxito", f"Datos de '{ename}' actualizados.")
                limpiar_campos(); consulta_empleados()
            else:
                messagebox.showinfo("Sin cambios", "No se detectaron cambios. Modifica al menos un campo.")
    except Exception as e:
        messagebox.showerror("Error", str(e))

def baja_empleado():
    if not driver_ok(): return

```

```

ename = ent_name.get().strip().upper()
if not ename:
    messagebox.showwarning("Faltan datos", "Escribe el nombre del empleado a eliminar.")
    return
if not solo_latras(ename, "Nombre"): return

try:
    with driver.session(database=DB_NAME) as session:
        emp = session.run(
            "MATCH (e:Emp {ename:$ename})-[:TRABAJA_EN]->(d:Dept) "
            "RETURN e, d.deptno AS deptno, d.dname AS dname", ename=ename).single()
        if not emp:
            messagebox.showwarning("No encontrado", f"No existe ningún empleado llamado '{ename}'.")
            return

        deptno = emp["deptno"]
        dname = emp["dname"]

        subordinados = session.run("MATCH (e:Emp {mgr:$ename}) RETURN e.ename AS nombre", ename=ename).values("nombre")
        if subordinados:
            lista = ", ".join([s[0] for s in subordinados])
            messagebox.showwarning("Empleado es jefe", f"'{ename}' es jefe de: {lista}\n\nActualiza el campo MGR de esos empleados antes de eliminar.")
            return

        conteo = session.run("MATCH (e:Emp)-[:TRABAJA_EN]->(d:Dept {deptno:$deptno}) RETURN count(e) AS n", deptno=deptno).single()[0]
        es_ultimo = (conteo == 1)

    except Exception as e:
        messagebox.showerror("Error", str(e))
        return

    if not messagebox.asksyesno("Confirmar", f"¿Seguro que deseas eliminar a '{ename}'?"): return

    borrar_dept = False
    if es_ultimo:
        borrar_dept = messagebox.asksyesno("Departamento vacío", f"'{ename}' es el único empleado en '{dname}'.\n\n¿Deseas eliminar también el departamento?")

```

```

try:
    with driver.session(database=DB_NAME) as session:
        session.run("MATCH (e:Emp {ename:$ename}) DETACH DELETE e", ename=ename)
        if borrar_dept:
            session.run("MATCH (d:Dept {deptno:$deptno}) WHERE NOT (d)-[:TRABAJA_EN]->() DELETE d", deptno=deptno)
            messagebox.showinfo("Éxito", f"Empleado '{ename}' y departamento '{dname}' eliminados.")
            consulta_departamentos()
        else:
            messagebox.showinfo("Éxito", f"Empleado '{ename}' eliminado.")
            limpiar_campos(); consulta_empleados()
    except Exception as e:
        messagebox.showerror("Error", str(e))

```

Funciones auxiliares de interfaz

seleccionar_fila(event): Cuando el usuario hace clic sobre una fila de la tabla de empleados, esta función copia los datos seleccionados a los campos del formulario para poder revisarlos o editarlos.

deseleccionar_tabla(event): Permite reactivar los campos de departamento y ubicación cuando el usuario deja de seleccionar una fila.

limpiar_campos(): Borra todos los campos de entrada y deja el formulario listo para capturar un nuevo registro.

cerrar_app(): Cierra la conexión con Neo4j y después destruye la ventana principal.

```

def seleccionar_fila(event):
    sel = tabla.focus()
    if not sel: return
    v = tabla.item(sel, "values")
    limpiar_campos()
    ent_empno.insert(0, "" if v[0] == "-" else v[0])
    ent_name.insert(0, v[1])
    ent_job.insert(0, v[2])
    ent_mgr.insert(0, "" if v[3] == "-" else v[3])
    ent_hiredat.insert(0, "" if v[4] == "-" else v[4])
    ent_sal.insert(0, v[5].replace("$", ""))
    ent_comm.insert(0, v[6].replace("$", ""))
    ent_dept.insert(0, v[7])

    ent_loc.configure(state="normal")
    ent_loc.insert(0, "" if v[8] == "-" else v[8])
    ent_loc.configure(state="disabled")

    ent_dept.configure(state="disabled")

def deseleccionar_tabla(event):
    ent_dept.configure(state="normal")
    ent_loc.configure(state="normal")

def limpiar_campos():
    ent_dept.configure(state="normal")
    ent_loc.configure(state="normal")
    for c in (ent_empno, ent_name, ent_job, ent_mgr, ent_hiredat,
              ent_sal, ent_comm, ent_dept, ent_loc, ent_buscar):
        c.delete(0, "end")
    lbl_total.configure(text="")

def cerrar_app():
    if driver: driver.close()
    app.destroy()

```

Interfaz gráfica

La interfaz se organiza en dos partes principales:

Panel izquierdo

Contiene dos pestañas:

Pestaña "Empleados"

Incluye campos para capturar:

- EMPNO
- Nombre
- Puesto
- Jefe

- Fecha de contratación
- Salario
- Comisión
- Departamento
- Ubicación de referencia

También contiene botones para:

- Alta
- Baja
- Actualizar
- Limpiar

Pestaña “Departamentos”

Incluye campos para registrar un nuevo departamento:

- DEPTNO
- Nombre
- Localización

Además, muestra una tabla con todos los departamentos registrados.

Panel derecho

Contiene la tabla principal de empleados y un área de búsqueda.

Elementos principales:

- Selector del criterio de búsqueda.
- Campo para escribir el valor a buscar.
- Botón Buscar.
- Botón Ver Todos.
- Etiqueta con el total de resultados.
- Tabla con los datos del empleado y su departamento.

Estilos visuales

El programa también configura el estilo de la tabla Treeview para que combine con el tema oscuro de la interfaz:

- fondo oscuro
- texto blanco
- encabezados con color azul
- fila seleccionada resaltada

Esto mejora la presentación y hace que los datos sean más fáciles de leer.

Flujo general del programa

1. Se crea la ventana principal.
2. Se intenta conectar a Neo4j.
3. Se cargan datos iniciales si la base está vacía.
4. Se muestran empleados y departamentos en sus respectivas tablas.
5. El usuario puede:
 - registrar departamentos,
 - registrar empleados,
 - buscar empleados,
 - actualizar datos,
 - eliminar registros.

Cuando se cierra la ventana, la conexión con Neo4j también se cierra correctamente.

Ejecución del programa

EmpleadosDepartamentos

Datos del Empleado

EMPNO (clave numérica):

Nombre (ENAME):

Puesto (JOB):

Jefe (MGR nombre):

Fecha contrato (AAAA-MM-DD):

Salario (SAL):

Comisión (COMM):

Departamento (DEPTNO):

Ubicación Referencia (LOC):

Alta

Baja

Actualizar

Limpiar

Nombre

BuscarVer Todos

Total: 14 empleado(s)

EMPNO	Nombre	Puesto	MGR	Contratado	Salario	Comisión	DEPTNO	LOC
7521	CASTILLO	VENDEDOR	MENDOZA	1981-02-22	\$1250.00	\$500.00	30	MONTERREY
7634	CRUZ	EMPLEADO	HERRERA	1982-01-23	\$1300.00	\$0.00	10	CIUDAD DE MEXICO
7600	FLORES	EMPLEADO	MENDOZA	1981-12-03	\$950.00	\$0.00	30	MONTERREY
7676	GARCIA	EMPLEADO	TORRES	1983-01-12	\$1100.00	\$0.00	20	GUADALAJARA
7469	GUTIERREZ	VENDEDOR	MENDOZA	1981-02-20	\$1600.00	\$300.00	30	MONTERREY
7782	HERRERA	GERENTE	REYES	1981-06-09	\$2450.00	\$0.00	10	CIUDAD DE MEXICO
7844	JIMENEZ	VENDEDOR	MENDOZA	1981-09-08	\$1500.00	\$0.00	30	MONTERREY
7569	LOPEZ	EMPLEADO	VARGAS	1980-12-17	\$800.00	\$0.00	20	GUADALAJARA
7698	MENDOZA	GERENTE	REYES	1981-05-01	\$2850.00	\$0.00	30	MONTERREY
7654	MORALES	VENDEDOR	MENDOZA	1981-09-28	\$1250.00	\$1400.00	30	MONTERREY
7566	RAMIREZ	GERENTE	REYES	1981-04-02	\$2975.00	\$0.00	20	GUADALAJARA
7839	REYES	DIRECTOR	-	1981-11-17	\$5000.00	\$0.00	10	CIUDAD DE MEXICO
7788	TORRES	ANALISTA	RAMIREZ	1982-12-09	\$3000.00	\$0.00	20	GUADALAJARA
7902	VARGAS	ANALISTA	RAMIREZ	1981-12-03	\$3000.00	\$0.00	20	GUADALAJARA

EmpleadosDepartamentos

Datos del Empleado

EMPNO (clave numérica):

Nombre (ENAME):

Puesto (JOB):

Jefe (MGR nombre):

Fecha contrato (AAAA-MM-DD):

Salario (SAL):

Comisión (COMM):

Departamento (DEPTNO):

Ubicación Referencia (LOC):

Alta

Baja

Actualizar

Limpiar

Nombre

BuscarVer Todos

Total: 14 empleado(s)

EMPNO	Nombre	Puesto	MGR	Contratado	Salario	Comisión	DEPTNO	LOC
7521	CASTILLO	VENDEDOR	MENDOZA	1981-02-22	\$1250.00	\$500.00	30	MONTERREY
7634	CRUZ	EMPLEADO	HERRERA	1982-01-23	\$1300.00	\$0.00	10	CIUDAD DE MEXICO
7600	FLORES	EMPLEADO	MENDOZA	1981-12-03	\$950.00	\$0.00	30	MONTERREY
7676	GARCIA	EMPLEADO	TORRES	1983-01-12	\$1100.00	\$0.00	20	GUADALAJARA
7469	GUTIERREZ	VENDEDOR	MENDOZA	1981-02-20	\$1600.00	\$300.00	30	MONTERREY
7782	HERRERA	GERENTE	REYES	1981-06-09	\$2450.00	\$0.00	10	CIUDAD DE MEXICO
7844	JIMENEZ	VENDEDOR	MENDOZA	1981-09-08	\$1500.00	\$0.00	30	MONTERREY
7569	LOPEZ	EMPLEADO	VARGAS	1980-12-17	\$800.00	\$0.00	20	GUADALAJARA
7698	MENDOZA	GERENTE	REYES	1981-05-01	\$2850.00	\$0.00	30	MONTERREY
7654	MORALES	VENDEDOR	MENDOZA	1981-09-28	\$1250.00	\$1400.00	30	MONTERREY
7566	RAMIREZ	GERENTE	REYES	1981-04-02	\$2975.00	\$0.00	20	GUADALAJARA
7839	REYES	DIRECTOR	-	1981-11-17	\$5000.00	\$0.00	10	CIUDAD DE MEXICO
7788	TORRES	ANALISTA	RAMIREZ	1982-12-09	\$3000.00	\$0.00	20	GUADALAJARA
7902	VARGAS	ANALISTA	RAMIREZ	1981-12-03	\$3000.00	\$0.00	20	GUADALAJARA

Exit

Empleado 'JUANITO' registrado en el departamento 'CONTABILIDAD'.

Aceptar

EmpleadosDepartamentos

Datos del Empleado

EMPNO (clave numérica):

Nombre (ENAME):

Puesto (JOB):

Jefe (MGR nombre):

Fecha contrato (AAAA-MM-DD):

Salario (SAL):

Comisión (COMM):

Departamento (DEPTNO):

Ubicación Referencia (LOC):

Alta

Baja

Actualizar

Limpiar

Nombre

Buscar

Ver Todos

EMPNO	Nombre	Puesto	MGR	Contratado	Salario	Comisión	DEPTNO	LOC
7521	CASTILLO	VENDEDOR	MENDOZA	1981-02-22	\$1250.00	\$300.00	30	MONTERREY
7934	CRUZ	EMPLEADO	HERRERA	1982-01-23	\$1300.00	\$0.00	10	CIUDAD DE MEXICO
7900	FLORES	EMPLEADO	MENDOZA	1981-12-03	\$950.00	\$0.00	30	MONTERREY
7876	GARCIA	EMPLEADO	TORRES	1983-01-12	\$1100.00	\$0.00	20	GUADALAJARA
7499	GUTIERREZ	VENDEDOR	MENDOZA	1981-02-20	\$1600.00	\$300.00	30	MONTERREY
7782	HERRERA	GERENTE	REYES	1981-06-09	\$2450.00	\$0.00	10	CIUDAD DE MEXICO
7844	JIMENEZ	VENDEDOR	MENDOZA	1981-09-08	\$1500.00	\$0.00	30	MONTERREY
1	JUANITO	EMPLEADO	TORRES	1990-05-16	\$1400.00	\$200.00	10	CIUDAD DE MEXICO
7369	LOPEZ	EMPLEADO	VARGAS	1980-12-17	\$800.00	\$0.00	20	GUADALAJARA
7988	MENDOZA	GERENTE	REYES	1981-05-01	\$2850.00	\$0.00	30	MONTERREY
7654	MORALES	VENDEDOR	MENDOZA	1981-09-28	\$1250.00	\$1400.00	30	MONTERREY
7566	RAMIREZ			1981-04-02	\$2975.00	\$0.00	20	GUADALAJARA
7839	REYES			1981-11-17	\$5000.00	\$0.00	10	CIUDAD DE MEXICO
7788	TORRES			1982-12-09	\$3000.00	\$0.00	20	GUADALAJARA
7902	VARGAS			1981-12-03	\$3000.00	\$0.00	20	GUADALAJARA

Exit

Datos de 'JUANITO' actualizados.

Aceptar

7844	JIMENEZ	VENDEDOR	MENDOZA	1981-09-08	\$1500.00	\$0.00	30	MONTERREY
1	JUANITO	EMPLEADO	TORRES	1990-05-16	\$1400.00	\$200.00	10	CIUDAD DE MEXICO
7369	LOPEZ	EMPLEADO	VARGAS	1980-12-17	\$800.00	\$0.00	20	GUADALAJARA

EmpleadosDepartamentos

Datos del Empleado

EMPNO (clave numérica):

Nombre (ENAME):

Puesto (JOB):

Jefe (MGR nombre):

Fecha contrato (AAAA-MM-DD):

Salario (SAL):

Comisión (COMM):

Departamento (DEPTNO):

Ubicación Referencia (LOC):

Alta

Baja

Actualizar

Limpiar

Nombre

Buscar

Ver Todos

EMPNO	Nombre	Puesto	MGR	Contratado	Salario	Comisión	DEPTNO	LOC
7521	CASTILLO	VENDEDOR	MENDOZA	1981-02-22	\$1250.00	\$300.00	30	MONTERREY
7934	CRUZ	EMPLEADO	HERRERA	1982-01-23	\$1300.00	\$0.00	10	CIUDAD DE MEXICO
7900	FLORES	EMPLEADO	MENDOZA	1981-12-03	\$950.00	\$0.00	30	MONTERREY
7876	GARCIA	EMPLEADO	TORRES	1983-01-12	\$1100.00	\$0.00	20	GUADALAJARA
7499	GUTIERREZ	VENDEDOR	MENDOZA	1981-02-20	\$1600.00	\$300.00	30	MONTERREY
7782	HERRERA	GERENTE	REYES	1981-06-09	\$2450.00	\$0.00	10	CIUDAD DE MEXICO
7844	JIMENEZ	VENDEDOR	MENDOZA	1981-09-08	\$1500.00	\$0.00	30	MONTERREY
1	JUANITO	EMPLEADO	TORRES	1990-05-16	\$1400.00	\$200.00	10	CIUDAD DE MEXICO
7369	LOPEZ	EMPLEADO	VARGAS	1980-12-17	\$800.00	\$0.00	20	GUADALAJARA
7988	MENDOZA	GERENTE	REYES	1981-05-01	\$2850.00	\$0.00	30	MONTERREY
7654	MORALES	VENDEDOR	MENDOZA	1981-09-28	\$1250.00	\$1400.00	30	MONTERREY
7566	RAMIREZ			1981-04-02	\$2975.00	\$0.00	20	GUADALAJARA
7839	REYES			1981-11-17	\$5000.00	\$0.00	10	CIUDAD DE MEXICO
7788	TORRES			1982-12-09	\$3000.00	\$0.00	20	GUADALAJARA
7902	VARGAS			1981-12-03	\$3000.00	\$0.00	20	GUADALAJARA

Exit

Empleado 'JUANITO' eliminado.

Aceptar

7844	JIMENEZ	VENDEDOR	MENDOZA	1981-09-08	\$1500.00	\$0.00	30	MONTERREY
7369	LOPEZ	EMPLEADO	VARGAS	1980-12-17	\$800.00	\$0.00	20	GUADALAJARA

EmpleadosDepartamentos

Nuevo Departamento

DEPTNO (clave numérica):

40

Nombre (DNAME):

test

Localización (LOC):

prueba

Registrar Depto

Departamentos Registrados

DEPTNO	DNAME	LOC
10	CONTABILIDAD	CIUDAD DE MEXICO
20	INVESTIGACION	GUADALAJARA
30	VENTAS	MONTERREY
40	OPERACIONES	CHIHUAHUA

Nombre

BuscarVer Todos

Total: 14 empleado(s)

EMPNO	Nombre	Puesto	MGR	Contratado	Salario	Comisión	DEPTNO	LOC
7521	CASTILLO	VENDEDOR	MENDOZA	1981-02-22	\$1250.00	\$300.00	30	MONTERREY
7534	CRUZ	EMPLEADO	HERRERA	1982-01-23	\$1300.00	\$0.00	10	CIUDAD DE MEXICO
7560	FLORES	EMPLEADO	MENDOZA	1981-12-03	\$950.00	\$0.00	30	MONTERREY
7576	GARCIA	EMPLEADO	TORRES	1983-01-12	\$1100.00	\$0.00	20	GUADALAJARA
7499	GUTIERREZ	VENDEDOR	MENDOZA	1981-02-20	\$1600.00	\$300.00	30	MONTERREY
7732	HERRERA	GERENTE	REYES	1981-06-09	\$2450.00	\$0.00	10	CIUDAD DE MEXICO
7844	JIMENEZ	VENDEDOR	MENDOZA	1981-09-08	\$1500.00	\$0.00	30	MONTERREY
7369	LOPEZ	EMPLEADO	VARGAS	1980-12-17	\$800.00	\$0.00	20	GUADALAJARA
7698	MENDOZA	GERENTE	REYES	1981-05-01	\$2850.00	\$0.00	30	MONTERREY
7654	MORALES	VENDEDOR	MENDOZA	1981-09-28	\$1250.00	\$1400.00	30	MONTERREY
7366	RAMIREZ	GERENTE	REYES	1981-04-02	\$2975.00	\$0.00	20	GUADALAJARA
7839	REYES	DIRECTOR	-	1981-11-17	\$5000.00	\$0.00	10	CIUDAD DE MEXICO
7788	TORRES	ANALISTA	RAMIREZ	1982-12-09	\$3000.00	\$0.00	20	GUADALAJARA
7902	VARGAS	ANALISTA	RAMIREZ	1981-12-03	\$3000.00	\$0.00	20	GUADALAJARA

EmpleadosDepartamentos

Nuevo Departamento

DEPTNO (clave numérica):

40

Nombre (DNAME):

test

Localización (LOC):

prueba

Registrar Depto

Departamentos Registrados

DEPTNO	DNAME	LOC
10	CONTABILIDAD	CIUDAD DE MEXICO
20	INVESTIGACION	GUADALAJARA
30	VENTAS	MONTERREY
40	OPERACIONES	CHIHUAHUA

Nombre

BuscarVer Todos

Total: 14 empleado(s)

EMPNO	Nombre	Puesto	MGR	Contratado	Salario	Comisión	DEPTNO	LOC
7521	CASTILLO	VENDEDOR	MENDOZA	1981-02-22	\$1250.00	\$300.00	30	MONTERREY
7534	CRUZ	EMPLEADO	HERRERA	1982-01-23	\$1300.00	\$0.00	10	CIUDAD DE MEXICO
7560	FLORES	EMPLEADO	MENDOZA	1981-12-03	\$950.00	\$0.00	30	MONTERREY
7576	GARCIA	EMPLEADO	TORRES	1983-01-12	\$1100.00	\$0.00	20	GUADALAJARA
7499	GUTIERREZ	VENDEDOR	MENDOZA	1981-02-20	\$1600.00	\$300.00	30	MONTERREY
7732	HERRERA	GERENTE	REYES	1981-06-09	\$2450.00	\$0.00	10	CIUDAD DE MEXICO
7844	JIMENEZ	VENDEDOR	MENDOZA	1981-09-08	\$1500.00	\$0.00	30	MONTERREY
7369	LOPEZ	EMPLEADO	VARGAS	1980-12-17	\$800.00	\$0.00	20	GUADALAJARA
7698	MENDOZA	GERENTE	REYES	1981-05-01	\$2850.00	\$0.00	30	MONTERREY
7654	MORALES	VENDEDOR	MENDOZA	1981-09-28	\$1250.00	\$1400.00	30	MONTERREY
7366	RAMIREZ	GERENTE	REYES	1981-04-02	\$2975.00	\$0.00	20	GUADALAJARA
7839	REYES	DIRECTOR	-	1981-11-17	\$5000.00	\$0.00	10	CIUDAD DE MEXICO
7788	TORRES	ANALISTA	RAMIREZ	1982-12-09	\$3000.00	\$0.00	20	GUADALAJARA
7902	VARGAS	ANALISTA	RAMIREZ	1981-12-03	\$3000.00	\$0.00	20	GUADALAJARA

Duplicado

El DEPTNO '40' ya está en uso.

Aceptar

Empleados

Departamentos

Nuevo Departamento

DEPTNO (clave numérica):

50

Nombre (DNAME):

test

Localización (LOC):

prueba

Registrar Depto

Departamentos Registrados

DEPTNO	DNAME	LOC
10	CONTABILIDAD	CIUDAD DE MEXICO
20	INVESTIGACION	GUADALAJARA
30	VENTAS	MONTERREY
40	OPERACIONES	CHIHUAHUA

Nombre

Buscar

Ver Todos

Total: 14 empleado(s)

EMPNO	Nombre	Puesto	MGR	Contratado	Salario	Comisión	DEPTNO	LOC
7321	CASTILLO	VENDEDOR	MENDOZA	1981-02-22	\$1250.00	\$300.00	30	MONTERREY
7934	CRUZ	EMPLEADO	HERRERA	1982-01-23	\$1300.00	\$0.00	10	CIUDAD DE MEXICO
7900	FLORES	EMPLEADO	MENDOZA	1981-12-03	\$950.00	\$0.00	30	MONTERREY
7876	GARCIA	EMPLEADO	TORRES	1983-01-12	\$1100.00	\$0.00	20	GUADALAJARA
7499	GUTIERREZ	VENDEDOR	MENDOZA	1981-02-20	\$1600.00	\$300.00	30	MONTERREY
7782	HERRERA	GERENTE	REYES	1981-06-09	\$2450.00	\$0.00	10	CIUDAD DE MEXICO
7844	JIMENEZ	VENDEDOR	MENDOZA	1981-09-08	\$1500.00	\$0.00	30	MONTERREY
7369	LOPEZ	EMPLEADO	VARGAS	1980-12-17	\$800.00	\$0.00	20	GUADALAJARA
7698	MENDOZA	GERENTE	REYES	1981-05-01	\$2850.00	\$0.00	30	MONTERREY
7604	MORALES	VENDEDOR	MENDOZA	1981-09-28	\$1250.00	\$1400.00	30	MONTERREY
7346	RAMIREZ	GERENTE	REYES	1981-04-02	\$2975.00	\$0.00	20	GUADALAJARA
7839	REYES	GERENTE		11-11	\$5000.00	\$0.00	10	CIUDAD DE MEXICO
7788	TORRES	GERENTE		12-09	\$2000.00	\$0.00	20	GUADALAJARA
7902	VARGAS	GERENTE		12-03	\$2000.00	\$0.00	20	GUADALAJARA

Exito

Departamento 'TEST' registrado correctamente.

Aceptar

Empleados

Departamentos

Nuevo Departamento

DEPTNO (clave numérica):

Nombre (DNAME):

Localización (LOC):

Registrar Depto

Departamentos Registrados

DEPTNO	DNAME	LOC
10	CONTABILIDAD	CIUDAD DE MEXICO
20	INVESTIGACION	GUADALAJARA
30	VENTAS	MONTERREY
40	OPERACIONES	CHIHUAHUA
50	TEST	PRUEBA

Link del repositorio:

https://github.com/AngelVillalobos-cloud/4.24_Proyecto-Final_BDADV